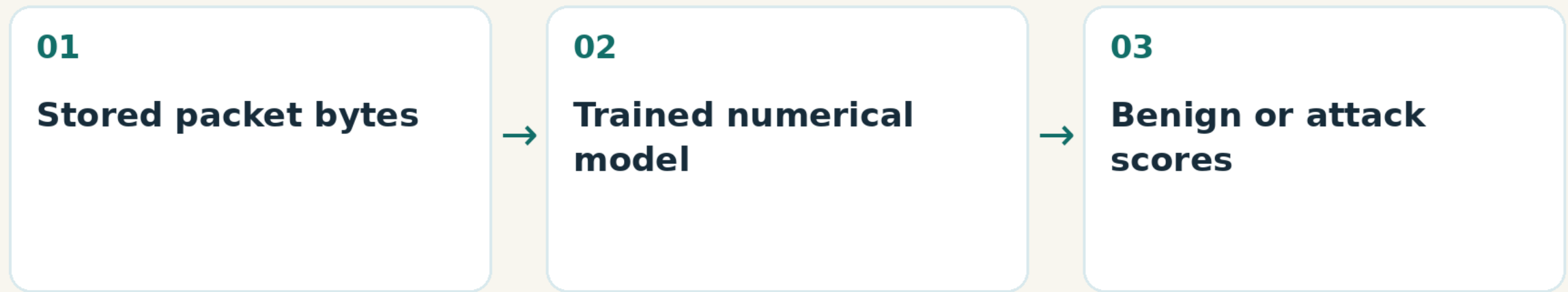


Follow one packet to a decision



Illustrative process · evidence and commands are linked in the transcript.

The paper, the implementation, and the evidence

Supplied file	Role in the project
2601.21792v1.pdf	Method to inspect; never model input
Payload_data_CICIDS2017.csv	CIC packet bytes and labels
Payload_data_UNSW.csv	UNSW packet bytes and labels

Paper: research method · Repository: executable work · Evidence: recorded outcomes

One supported route from both CSVs



Illustrative process · evidence and commands are linked in the transcript.

What joint pretrained means

- 01** Joint: examples from both sources
- 02** Pretrained: transferred starting weights
- 03** One selected packet classifier

Learning a rule and using a saved rule

01 Training changes weights

02 Checkpoint saves a model

03 Inference applies fixed weights

Two repositories, one packet workflow

Repository	Purpose
netmambaplus-reproduction	Development, research, evidence, media
netmambaplus-client	Setup, execution, tests, packet evidence

Full source and teaching record · Concise executable client delivery

Explain the delivered capability precisely

- 01** Runnable packet classification
- 02** Saved evidence and offline viewer
- 03** Performance tied to the tested exports

Explain it back

01 Explain the packet route and the purpose of each repository.

Speak or write your answer. The lesson resumes automatically.

Evidence: [project-guide](#) | [Links and full text in the transcript](#)

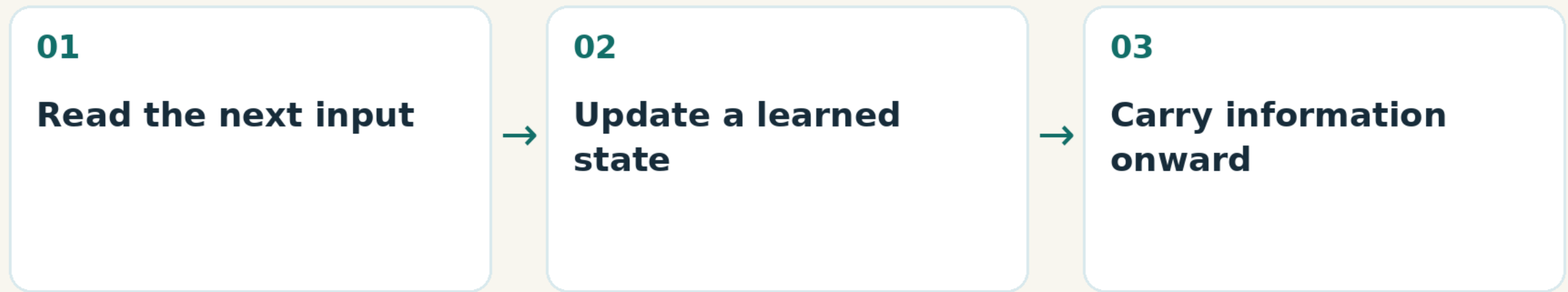
Three views of traffic in the paper

01 Bytes: recorded content

02 Sizes: packet lengths

03 Timing: arrival intervals

Mamba carries a numerical state forward



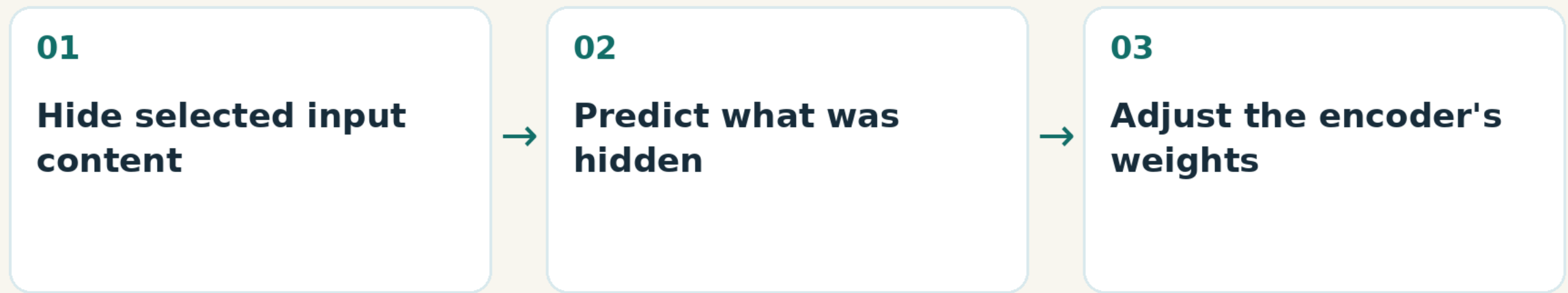
Illustrative process · evidence and commands are linked in the transcript.

Evidence: [paper-notes](#) · [upstream-comparison](#) · [packet-report](#) | Links and full text in the transcript

A packet row cannot establish a whole flow

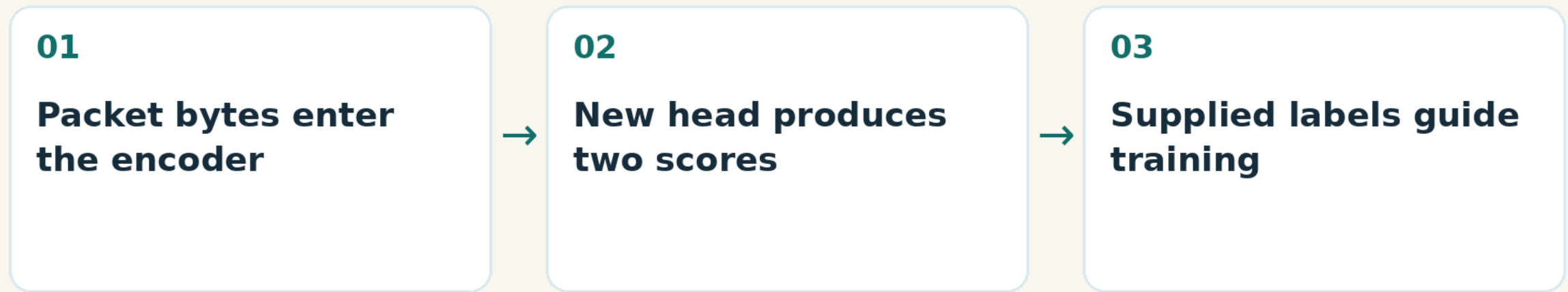
- 01** Packet: one network unit
- 02** Flow: related packets with defined order
- 03** Connection and ordering evidence are missing

Pretraining learns to reconstruct hidden content



Illustrative process · evidence and commands are linked in the transcript.

Supervised learning teaches the packet categories



Illustrative process · evidence and commands are linked in the transcript.

Evidence: [project-guide](#) · [packet-report](#) · [upstream-comparison](#) | Links and full text in the transcript

What the packet adaptation actually observes

- 01** All 1,500 stored payload bytes
- 02** No observed size or interval sequences
- 03** Metadata stays outside the neural call

A similar percentage is not the same experiment

01 Paper Table IV: 97.50% CICIoT2022 flow accuracy

02 Packet study: a different task and metric

03 Compare the conditions before the numbers

Explain it back

01 Why is this a packet adaptation rather than the paper's flow experiment?

Speak or write your answer. The lesson resumes automatically.

Evidence: [packet-report](#) · [paper-notes](#) | [Links and full text in the transcript](#)

A text table of packet examples

01 CICIDS2017 supplied export

02 1,410,255 packet rows

03 15 original labels

Read one row in three parts

Part	Contents
Payload	1,500 ordered byte slots
Metadata	ttl, total_len, protocol, t_delta
Target	label

One row represents one packet example · 1,505 columns in a fixed order

Byte slots contain numbers, including stored zeros

01 Integer values from 0 through 255

02 Exact order is preserved

03 Stored zeros remain in the input

Four recorded properties, with specific limits

Field	Recorded meaning
ttl	Time to live; network lifetime field
total_len	Recorded packet length
protocol	Traffic carrying protocol
t_delta	Time difference; interpretation unestablished

Profiled and checked for separate controls · Never inputs to the native neural call

Keep the answer outside the model input

- 01** BENIGN maps to benign, class 0
- 02** Registered attack labels map to class 1
- 03** Unknown labels fail validation

Many rows repeat the same stored payload

- 01** 1,410,255 rows
- 02** 438,886 distinct payload groups
- 03** Every original label count is retained

Check what the selected test set contains

01 Counts are not performance

02 No PortScan test rows

03 Four DDoS test rows

Explain it back

01 Explain the three parts of a row and which part reaches the model.

Speak or write your answer. The lesson resumes automatically.

UNSW uses the same row structure

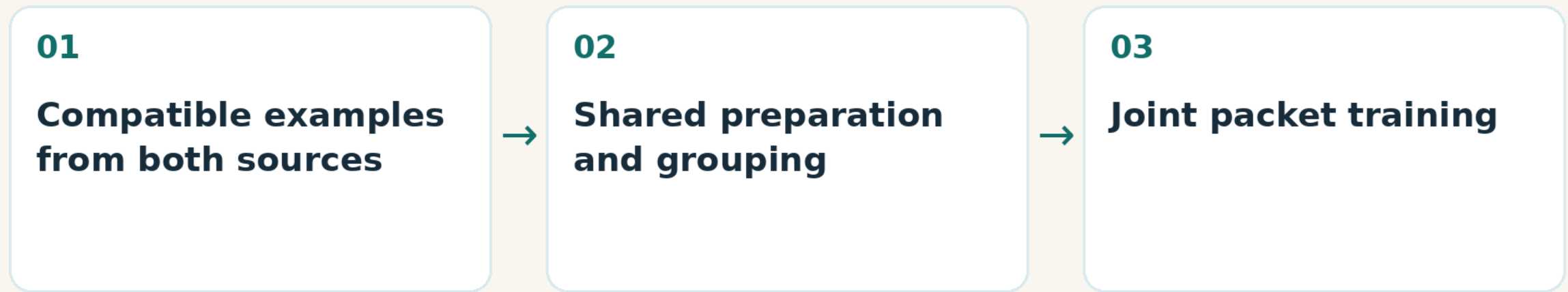
- 01** 79,881 packet rows
- 02** The same 1,505-column schema
- 03** 10 original labels

Map source labels to the shared binary task

Source	Benign	Attack
CICIDS2017	BENIGN	Other registered CIC labels
UNSW	normal	Other registered UNSW labels

Keep original source annotations · Use one benign or attack target

Combine packet examples without inventing connections



Illustrative process · evidence and commands are linked in the transcript.

Evidence: [packet-report](#) · [packet-data](#) · [packet-protocol](#) · [packet-manifest](#) | Links and full text in the transcript

Find exact payload duplicates across both sources



Illustrative process · evidence and commands are linked in the transcript.

Evidence: [packet-data](#) · [packet-report](#) · [project-guide](#) · [packet-manifest](#) | Links and full text in the transcript

Shared payloads can carry conflicting answers

- 01** 10 payloads appear in both files
- 02** 9 of those have conflicting binary labels
- 03** Disagreements remain in the evidence

Preserve disagreement as label proportions

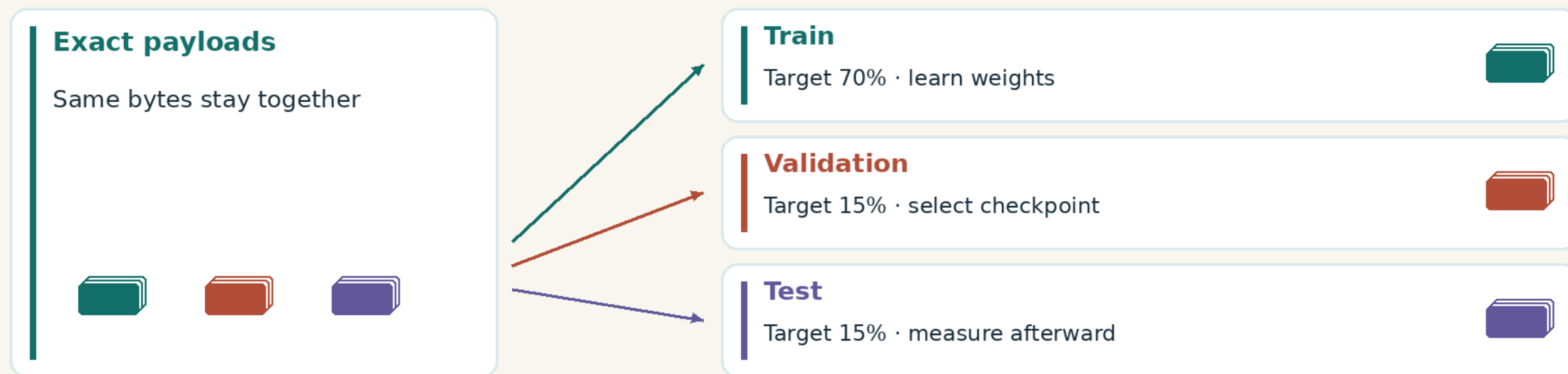
01 Illustration: nine benign rows, one attack row

02 Training target: 90% benign, 10% attack

03 Each sampled group has equal total weight

Keep exact duplicates inside one partition

SPLIT GROUPS, KEEP DUPLICATES TOGETHER



Seed 17 · target proportions before caps · identical payloads stay in one split; capture independence unknown.

Evidence: [packet-data](#) · [packet-report](#) · [project-guide](#) · [packet-manifest](#) | Links and full text in the transcript

Explain it back

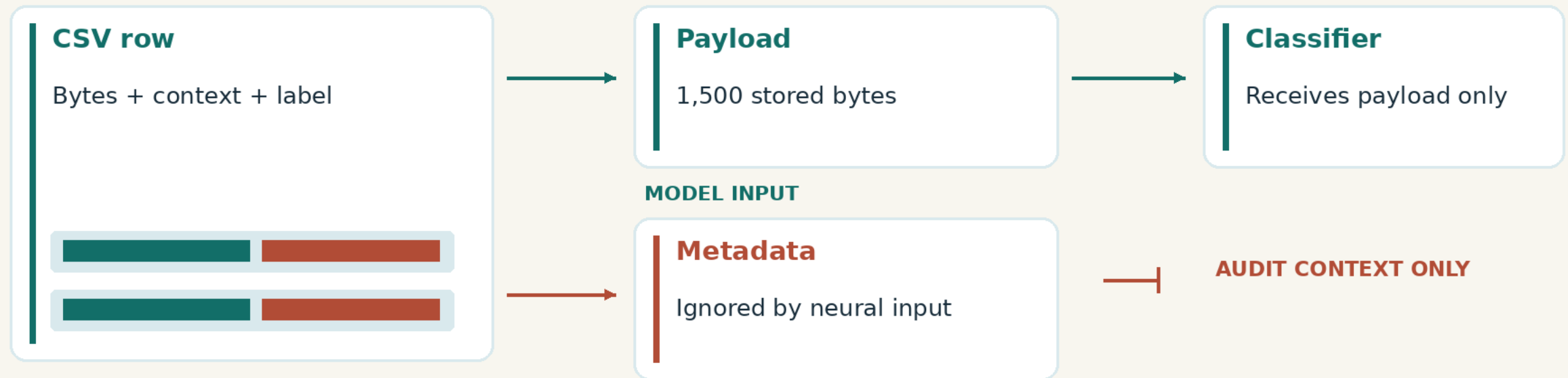
01 Why group both sources before splitting, and what uncertainty remains?

Speak or write your answer. The lesson resumes automatically.

Evidence: [packet-data](#) · [packet-report](#) · [packet-manifest](#) | [Links and full text in the transcript](#)

One packet supplies the model input

ONE ROW, SEPARATE RESPONSIBILITIES



Illustrative data path · supplied labels affect training loss, never model input.

Rescale the bytes without changing their order

```
batch = payload_tensor.shape[0]
byte_stream = (
    payload_tensor.to(dtype=torch.float32) / 127.5 - 1
).reshape(batch, 1, 1, 1500)
```

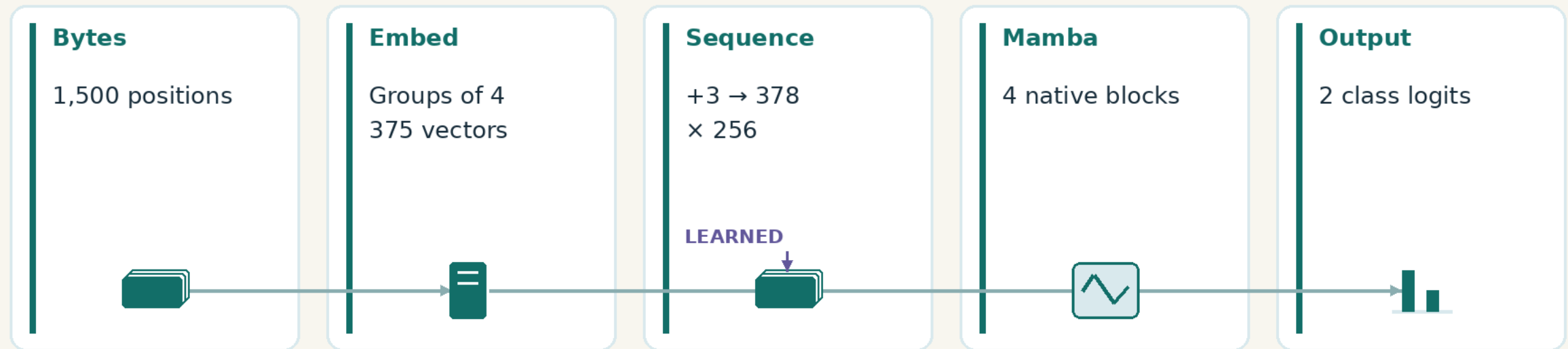
Byte 0 becomes -1 ; byte 255 becomes $+1$

Tensor: $\text{batch} \times 1 \times 1 \times 1,500$

Stored zeros remain in the input

Assemble 378 tokens from one packet

THE PACKET ENCODER, STAGE BY STAGE



Packet adaptation · 375 byte vectors + 3 learned tokens; empty size and timing observations.

Evidence: [packet-model](#) · [packet-report](#) | [Links and full text in the transcript](#)

Empty streams still have learned prefixes

- 01** Size observations: 0; interval observations: 0
- 02** Two learned prefixes begin identically for each packet
- 03** Prefix outputs cannot summarize later bytes

Four blocks lead to two class scores



Packet classifier total: 1,852,416 parameters.

Transfer compatible weights into the packet model

Component	Packet treatment
Compatible trunk tensors	50 copied
Reconstruction tensors	31 excluded
Binary head	Fresh weights
Position table	378 selected from 443; trainable

Reuse compatible encoder state; start a fresh binary head · Learned positions remain trainable

Position indices: [20, 41] + range(42, 417) + [442].

Explain the complete feature path

? Describe the stored bytes and their transformation

? Identify learned tokens and empty streams

? Separate model inputs from training answers

Your turn · 10 seconds

01 Explain how one packet becomes two scores, including what stays outside.

Speak or write your answer. The lesson resumes automatically.

Evidence: [packet-model](#) · [packet-report](#) | [Links and full text in the transcript](#)

Preserve disagreements in the learning target

01

**Same payload: 9
benign labels + 1
attack label**



02

**Soft target: 90%
benign / 10% attack**



03

**One sampled group
contributes one total
weight**

Illustrative conflicting group; every sampled group has equal total loss weight.

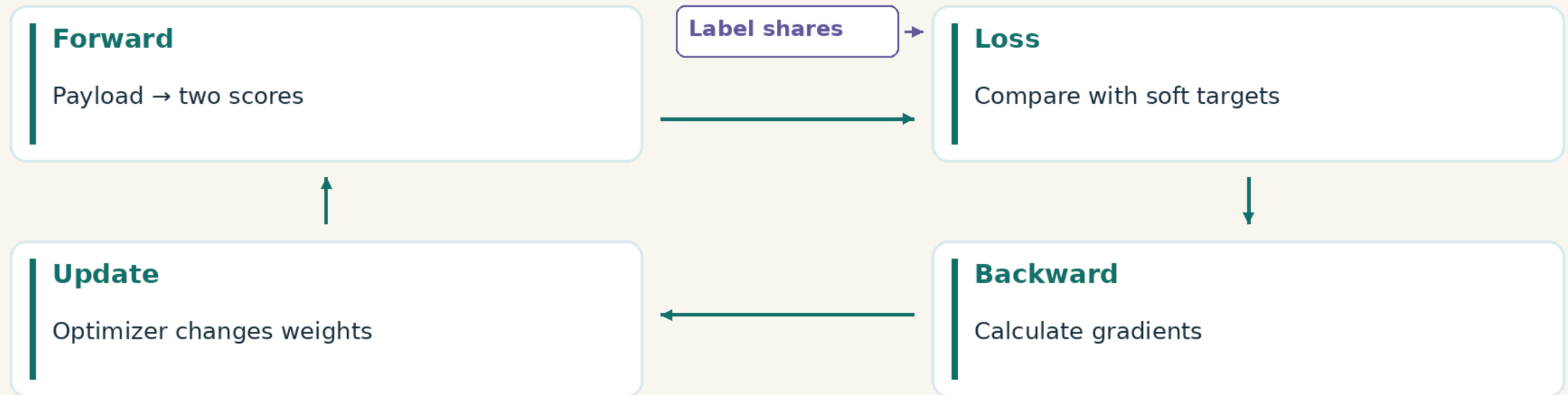
Compare six training arms

Training source	Transferred initialization	Fresh initialization
CIC only	cic_pretrained	cic_scratch
UNSW only	unsw_pretrained	unsw_scratch
CIC + UNSW	joint_pretrained	joint_scratch

Three source choices × two initializations · One model seed: 0 · Paired arms share fresh heads and sampled groups

One update is one learning step

WHAT CHANGES WHEN THE MODEL LEARNS



Recorded protocol: 1,000 updates per arm · batch 64 · replacement sampling.

Evidence: [packet-training](#) · [packet-protocol](#) · [packet-results](#) | Links and full text in the transcript

Caps and replacement sampling define exposure

Source	Selected training groups	Groups per joint batch
CIC	100,000	32
UNSW	27,380	32

Joint batches: 32 groups from each source · Sampling with replacement can repeat a group · 64,000 presentations per model, not full CSV epochs

Validation chooses; frozen test groups measure



Validation selects weights; the final tests do not select the checkpoint.

Freeze the experiment and retain its receipts

- 01** Protocol binds data, code, rules, and budget
- 02** Checkpoints retain selection and input identities
- 03** Receipts establish achieved optimizer updates

Distinguish training, selection, and inference

? Training groups change the weights

? Validation groups choose saved weights

? Inference applies a selected checkpoint

Your turn · 10 seconds

01 Explain which data changes weights, chooses a checkpoint, and measures it.

Speak or write your answer. The lesson resumes automatically.

Evidence: [packet-protocol](#) · [packet-training](#) | [Links and full text in the transcript](#)

Find the five implementation responsibilities

File	Responsibility
packet_data.py	Validate and prepare groups
packet_model.py	Native adapter and checkpoints
packet_study.py	Metrics and evidence utilities
tools/train_packet_model.py	Run the frozen training study
tools/predict_packets.py	Predict an unlabeled packet CSV

Follow data preparation through training to prediction · Shared metrics keep result arithmetic consistent

Evidence: [packet-data](#) · [packet-model](#) · [packet-training](#) · [packet-prediction](#) · [packet-review](#) | [Links and full text in the transcript](#)

The forward call receives only payload tensors

```
# payload: [batch, 1500], unsigned bytes
logits = packet_model.forward_payload(model, payload)['logits']
# logits: [batch, 2], floating point scores
```

Input: unsigned byte tensor,
batch $\times$ 1,500

Output: floating point logits,
batch $\times$ 2

Training targets stay outside this
call

Loss, gradients, and an optimizer update

```
optimizer.zero_grad(set_to_none=True)
logits = packet_model.forward_payload(model, payload)['logits']
loss = -(
    truth * torch.log_softmax(logits, dim=1)
).sum(dim=1).mean()
loss.backward()
optimizer.step()
```

Loss measures disagreement with soft targets

Gradients connect that loss to model parameters

The optimizer changes weights

Load the selected checkpoint for inference

01

Read Selected joint pretrained checkpoint

**02**

Strictly load weights and their metadata

**03**

Apply fixed weights to new packet bytes

The selected path is relative to its packet study directory.

Validate the CSV before publishing completion

Input layout	Accepted
1,500 ordered payload columns	Yes
Payload columns + ttl,total_len,protocol,t_delta	Yes; metadata stays outside forward
Any header containing label	No

Exact unlabeled header and valid byte cells are required · A prediction cap still scans the whole CSV · The receipt distinguishes validated rows from predictions

Full input validation precedes a successful completion receipt, not the first prediction.

Read class decisions and uncalibrated scores

- 01** logits: two finite raw scores
- 02** class_name: benign or attack
- 03** probabilities and receipt: scores plus run identity

Check native execution on the prepared packets

```
python tools/check_packet_runtime.py \  
  --data runs/packet-data-new \  
  --upstream upstream/NetMambaPlus \  
  --initialization assets/checkpoints/fuse3_mamba.pth \  
  --output runs/packet-runtime-new
```

17 native GB10 tests passed; 0 skipped

Stored payloads with synthetic targets test functionality

Your turn · 10 seconds

01 Explain what a successful capped prediction receipt proves about the input.

Speak or write your answer. The lesson resumes automatically.

Evidence: packet-prediction · native-gate | Links and full text in the transcript

The selected joint model's measured results

Test source	Distinct groups	Group balanced accuracy
CIC	20,000	97.56%
UNSW	5,930	94.29%

Joint pretrained checkpoint selected before final tests · 25,930 distinct test payload groups in total

All six native models were evaluated on these same source test groups.

Class balance and duplicate weighting answer different things

Joint pretrained test	Group balanced accuracy	Row balanced accuracy
CIC	97.56%	96.31%
UNSW	94.29%	89.24%

Balanced accuracy averages benign and attack recall · Group weight: one total vote per distinct payload · Row weight: one vote per original row

Pretrained models show transfer tradeoffs

Pretrained training source	CIC test	UNSW test
CIC only	98.67%	53.73%
UNSW only	71.52%	93.22%
Joint	97.56%	94.29%

Each row is one model; each column is a test source · Joint versus source models: +1.07 points UNSW, −1.11 CIC

Every score in this table is group weighted balanced accuracy.

All scratch arms trained; their outcomes were weak

Scratch training source	Selected update	CIC test	UNSW test
CIC only	100	50.00%	50.00%
UNSW only	100	50.00%	50.00%
Joint	1,000	55.57%	50.00%

Every scratch arm completed 1,000 updates · Single-source selections made constant-class decisions

Test scores are group weighted balanced accuracy.

Nine simple controls challenge neural superiority

- 01** 3 source choices × majority, histogram, and metadata
- 02** UNSW metadata: 99.34% group balanced accuracy there
- 03** Different input features are part of the comparison

Class agreement and numerical agreement are distinct

Check scope	Validated / predicted	Classes agree	Strict logit failures
Original full check	25,930 / 25,930	25,930	2 rows
Fresh client check	20,000 / 128	128	76 rows

Original full check and fresh capped check remain separate · Failures count compared prediction rows

Fresh max $|\Delta \text{logit}| = 0.006159305572509766$; $\text{rtol}=1\text{e-}4$, $\text{atol}=1\text{e-}6$.

State what the evidence still cannot establish

? One model seed, capped pools, fixed update budget

? Earlier pretrained exposure remains unknown

? CIC test: 0 PortScan rows; only 4 DDoS rows

Your turn · 10 seconds

01 State one measured success, one weak result, and two remaining limits.

Speak or write your answer. The lesson resumes automatically.

Evidence: [packet-report](#) · [packet-results](#) · [client-audit](#) | [Links and full text in the transcript](#)

Start with the right demonstration

01 Recorded joint-model predictions

02 Both packet sources

03 No live network connection

Choose the source you are explaining

Test source

Show groups

Predicted class

Payload hash prefix

CICIDS2017

All groups

Both classes

e.g. 0000b8

Balanced accuracy

97.56%

Group weighting · complete source test

Distinct payload groups

20,000

47,048 original rows represented

Label disagreement

608

Groups with at least one mismatched label

Conflicting labels

2

Same bytes labeled both benign and attack

Inspect the evidence

20,000 matching groups from 20,000 in CICIDS2017. 1,656 original rows disagree with the model across this complete source test.

Each row below is one distinct payload group. Label counts are original CSV rows, not model scores. Select a hash to inspect logits.

Payload hash	Prediction	Model score	Benign labels	Attack labels	Mismatched rows
0000b88b75f3ff_	Benign	99.81%	1	0	0
0002d34b621085_	Benign	85.34%	1	0	0
000334d80349fa_	Attack	99.76%	0	8	0
0003979053adc5_	Attack	98.83%	0	5	0
00065821ad3111_	Attack	98.54%	0	1	0

CIC: 20,000 test groups

UNSW: 5,930 test groups

Headline metrics use the complete source

One decision has several parts

01 Payload hash identifies the byte group

02 Two logits determine the class

03 Scores are uncalibrated

Keep the supplied labels visible

- 01** Original benign and attack row counts
- 02** Conflicting labels remain visible
- 03** A group is not automatically one true label

Demonstrate an error without hiding it

Payload hash	Prediction	Model score	Benign labels	Attack labels	Mismatched rows
0062ae63cce54d...	Attack	94.57%	1	0	1

Actual saved error: Attack, 94.57% uncalibrated score · Supplied labels: 1 benign, 0 attack · A prediction is not an independently verified incident

Fresh prediction is a separate execution step

- 01** Supply an unlabeled packet CSV
- 02** Load the selected joint checkpoint
- 03** Retain predictions and the receipt

What a working IDS would add

01 Capture and compatible byte extraction

02 Inference queues and alert policy

03 Monitoring, review and response

Your turn: explain it aloud

01 Explain what the browser demonstrates and what requires the GPU predictor.

Speak or write your answer. The lesson resumes automatically.

Evidence: [project-guide](#) | [Links and full text in the transcript](#)

Start from the client repository

```
git clone https://github.com/buffbeefalo/netmambaplus-client.git
cd netmambaplus-client
python -m unittest discover -s tests -v
python tools/verify_client.py
```

Clone netmambaplus-client

Read SETUP.md from its root

Keep output directories separate

Check the portable parts first

```
python -m unittest discover -s tests -v
python tools/verify_client.py
python tools/render_packet_demo.py \
  --output runs/packet-demo-new
```

Python 3.10 or 3.12

Tests and saved-evidence
verification

Offline packet viewer

Prepare the native execution environment

- 01** Measured: Linux ARM64 and GB10
- 02** Torch plus CUDA toolkit and custom operators
- 03** Other GPUs need their own checks

Acquire only the required initialization

```
python repro.py fetch
python tools/fetch_assets.py --output assets
python tools/build_cuda.py \
  --build-root runs/cuda-build-new
python tools/check_gpu_runtime.py \
  --output runs/cuda-ops-new.json
```

Verify the pinned authors' source

Fetch pretrained weights separately

Supply both CSVs locally

Prepare both inputs and run the native gate

- 01** Validate and group both CSVs
- 02** Check registered source identities
- 03** Require native tests with no skips

Freeze, train, select and predict

01 Freeze the protocol before the study

02 Complete the six comparison arms

03 Use the joint pretrained checkpoint

Read a failed check before retrying

01 Find the failing stage and receipt

02 Correct the cause in a new run

03 Never turn a skip into a pass

Your turn: explain it aloud

01 Name the evidence needed before claiming a new machine runs the native model.

Speak or write your answer. The lesson resumes automatically.

Evidence: project-guide | Links and full text in the transcript

Two repos, one packet application

01 Reproduction: develop, investigate and learn

02 Client: run the concise delivery

03 Shared code follows one workflow

The model and data files have distinct jobs

- 01** packet_data.py prepares inputs
- 02** packet_model.py adapts the encoder
- 03** packet_study.py supports the experiment

Tools, configs and requirements support execution

- 01** Tools run specific workflow stages
- 02** Configs record settings and identities
- 03** Requirements define the measured environment

Evidence and local runs are different objects

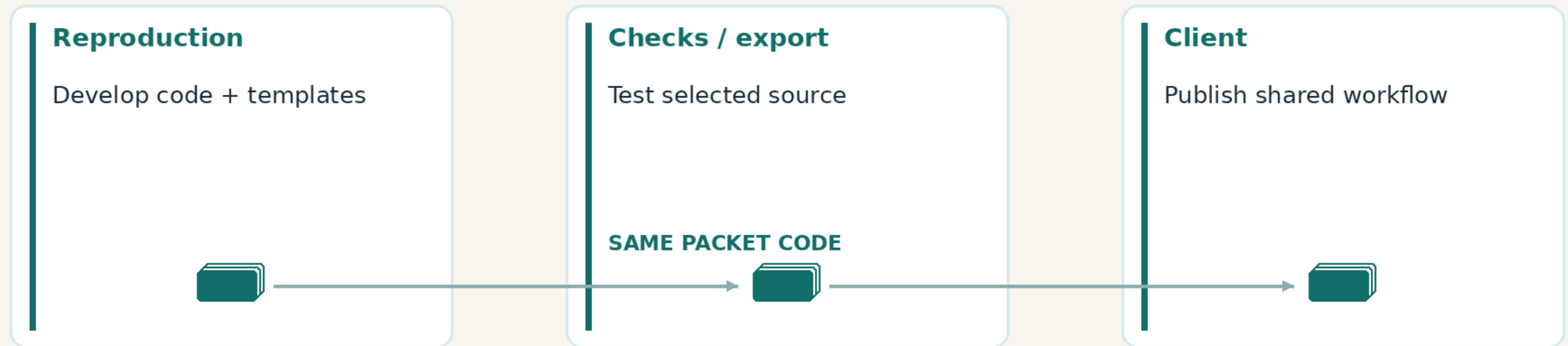
- 01** Public receipts and saved predictions
- 02** Local CSVs, prepared arrays and weights
- 03** Historical research stays labeled

The client manifest identifies the delivery

- 01** 124 files at the recorded delivery
- 02** 75 packet-study evidence files
- 03** Exact source paths and SHA-256 hashes

Updates flow from source to client

ONE PACKET IMPLEMENTATION, TWO REPOSITORIES



Source updates produce a verified client export; teaching media stays in reproduction.

Evidence: client-maintenance · client-audit | Links and full text in the transcript

Find the right download and its scope

01 Video, captions and transcript share a source

02 Slides and handbook follow the lesson

03 Client excludes teaching media

Your turn: explain it aloud

01 Explain which repo you would give a presenter and which you would give an engineer.

Speak or write your answer. The lesson resumes automatically.

Evidence: [project-guide](#) | [Links and full text in the transcript](#)

Start with the complete processing path

01 Capture and extract compatible bytes

02 Queue, infer and produce an event

03 Measure the whole path

What a SmartNIC could contribute

- 01** Network-facing capture and buffering
- 02** Possible preprocessing and queue handling
- 03** No completed target deployment

What an NPU port would require

- 01** A target runtime and supported operators
- 02** Equivalent tensor layouts and precision
- 03** Compare outputs against the GPU reference

Optimization needs a new measurement plan

- 01** Batching changes latency and throughput
- 02** Lower precision can change predictions
- 03** Compare accuracy, failures and resource use

The major research gaps remain visible

01 Independent customer traffic and more seeds

02 Unknown attacks and reliable confidence

03 Live extraction, alerts and target execution

Say precisely what has been built

- 01** Both CSVs really enter native training
- 02** The joint classifier predicts packet labels
- 03** The records include weak results and failures

Teach the complete story to the next person

- 01** Inputs, model, training and predictions
- 02** Both repos, tests and measured results
- 03** Limits and the next deployment experiment

Your turn: explain it aloud

- 01** Explain the complete CSV workflow, both repos, one measured result and one remaining limitation.

End of lesson. Explain it aloud, or replay a chapter to review.